

# CAS4160 Homework 7: RLHF

[Your name]  
[Your student ID]

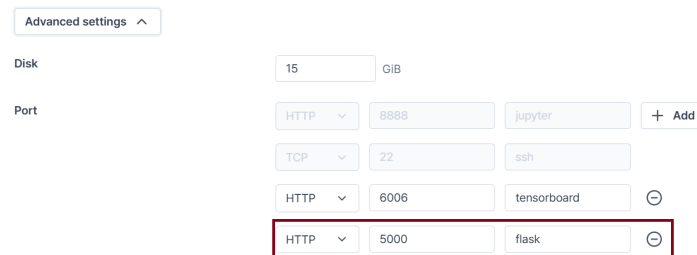
## 1 Introduction

이 과제의 목표는 인간 피드백을 통한 강화학습(Reinforcement Learning from Human Feedback, RLHF)을 이해하고 직접 실습해보는 것입니다. 기존의 강화학습 알고리즘이 미리 정의된 환경 보상(environment reward)에 의존하는 것과 달리, RLHF는 인간의 피드백으로부터 보상 모델(reward model)을 학습한 뒤, 학습된 보상을 사용하여 정책(policy)을 훈련합니다.

RLHF는 대규모 언어 모델(LLM) 학습에 사용된 것으로 널리 알려져 있지만, 실제 최초의 연구는 로봇 제어(robot control) 분야에서 수행되었습니다.

이번 과제에서는 이전에 개발했던 PPO 알고리즘을 확장하여 RLHF 프레임워크를 구현하게 됩니다. 환경이 제공하는 보상을 사용하는 대신, 인간의 선호(human preferences)를 기반으로 한 보상 함수를 정의하고, 이를 바탕으로 보상 모델을 학습시킬 것입니다. 이후 여러분이 직접 피드백을 제공하여 로봇의 행동을 유도하게 되며, 이를 통해 RLHF가 어떻게 동작하는지 실질적으로 이해하고, 로봇이 점점 발전하는 모습을 보며 재미도 느낄 수 있을 것입니다.

## 2 Installation



| Advanced settings ^ |      |      |               |
|---------------------|------|------|---------------|
| Disk                | 15   | GiB  |               |
| Port                | HTTP | 8888 | jupyter + Add |
|                     | TCP  | 22   | ssh           |
|                     | HTTP | 6006 | tensorboard ⊖ |
|                     | HTTP | 5000 | flask ⊖       |

Figure 1: Open port 5000 for Flask

이번 과제에서는 이전 과제들과 달리 추가적인 패키지 설치가 필요합니다. 시작하기 전에 반드시 `requirements.txt` 파일을 확인하고, 필요한 모든 의존성(dependencies)을 설치하시기 바랍니다.

또한, 피드백(선호도)을 제공하기 위해 Flask라는 웹 프레임워크를 사용할 예정입니다. 만약 VESSL AI를 사용하고 있다면, 웹 인터페이스가 정상적으로 동작할 수 있도록 추가 포트를 열어주어야 합니다. 자세한 내용은 Figure 1를 참고하시기 바랍니다. 또한 이 과정에서는 Chrome 브라우저를 사용해 주세요.

## 3 Code structure overview

코드 구조는 HW2와 유사하지만, 이번에는 RLHF를 위해 몇 가지 구성 요소가 추가됩니다. 이번 과제에서는 주로 다음 코드들을 다루게 됩니다.

- cas4160/networks/reward\_predictor.py: 보상 모델(reward model) 구현
- cas4160/agents/pg\_agent.py: 학습된 보상 모델을 사용하는 PPO 에이전트
- cas4160/scripts/run\_hw7.py: 구현을 위한 메인 학습 루프

## 4 Reinforcement learning from human feedback

먼저 인간 피드백을 통한 강화학습(Reinforcement Learning from Human Feedback, RLHF)에 대한 간단한 개요를 설명한 뒤, 이를 구현하게 됩니다. 여러분은 다음과 같은 과정을 수행하게 됩니다: (1) 보상 함수(reward function)를 정의하고 보상 학습 목표(reward learning objective)를 구현하며, (2) 사람이 주석(annotation)한 선호도(preference) 샘플을 수집하여 리플레이 버퍼(replay buffer)에 저장하고 보상 모델을 학습시키고, 마지막으로 (3) 학습된 보상을 사용하여 PPO 에이전트를 학습시킵니다.

### 4.1 Formulation of preference

우리는 에이전트가 여러 단계에 걸쳐 환경과 상호작용하는 상황을 고려한다. 각 시점  $t$ 에서 에이전트는 상태(state)  $\mathbf{s}_t \in \mathcal{S}$ 를 관찰하고 행동(action)  $\mathbf{a}_t \in \mathcal{A}$ 를 선택한다. 일반적인 강화학습(RL)에서는 환경이 스칼라 보상(scalar reward)  $r_t \in \mathbb{R}$ 를 반환하며, 에이전트의 목표는 기대 누적 할인 보상(expected cumulative discounted reward)을 최대화하는 것이다.

반면, 여기서는 환경이 명시적인 보상을 제공하지 않는다고 가정한다. 대신 인간 감독자(human overseer)가 짧은 trajectory 구간들에 대한 선호 비교(preference comparisons)를 제공한다. Trajectory 구간은 다음과 같은 시퀀스로 정의된다:

$$\tau = ((\mathbf{s}_0, \mathbf{a}_0), (\mathbf{s}_1, \mathbf{a}_1), \dots, (\mathbf{s}_{k-1}, \mathbf{a}_{k-1})).$$

우리는  $\tau_1 \succ \tau_2$ 를 통해 인간이 구간  $\tau_1$ 을  $\tau_2$ 보다 선호함을 나타낸다.

**목표 (Goal).** 에이전트는 인간이 선호하는 trajectory를 생성해야 하며, 동시에 선호도 질의(preference query)의 수를 최소화해야 한다.

**평가 기준 (Evaluation criteria).**

- **정량적 평가 (Quantitative evaluation):** 선호도가 알려지지 않은 보상 함수  $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 에 의해 유도된다고 가정하자. 그러면

$$\tau_1 \succ \tau_2$$

는 다음을 의미한다:

$$\sum_{t=0}^{k-1} r(\mathbf{s}_t^1, \mathbf{a}_t^1) > \sum_{t=0}^{k-1} r(\mathbf{s}_t^2, \mathbf{a}_t^2).$$

만약  $r$ 이 알려져 있다면, 실제 누적 보상(true cumulative rewards)을 기준으로 에이전트를 평가할 수 있다.

- **정성적 평가 (Qualitative evaluation):**  $r$ 이 알려져 있지 않은 경우에는, 에이전트의 행동이 인간의 선호와 얼마나 잘 일치하는지를 기반으로 평가한다. 이를 위해 인간에게 자연어 목표(natural language goal)를 제시하고, 관찰된 에이전트 행동 영상(agent behavior videos)을 바탕으로 평가를 요청한다.

우리의 설정에서는 인간 감독자에게 두 개의 trajectory 구간을 시각화한 짧은 영상 클립(short movie clips)이 제공되며, 각 영상의 길이는 1 ~ 2초 정도이다.

인간 주석자(human annotator)는 다음 중 하나를 선택하여 피드백을 제공한다:

- $\tau_1$ 이  $\tau_2$ 보다 더 선호된다.

- $\tau_2$ 가  $\tau_1$ 보다 더 선호된다.
- 두 구간 모두 동일하게 좋다.

각 비교 결과는 replay buffer  $\mathcal{D}$ 에 삼중항(triplet)  $(\tau_1, \tau_2, \mu)$  형태로 저장되며, 여기서:

- $\tau_1$ 과  $\tau_2$ 는 비교된 두 trajectory 구간이고,
- $\mu$ 는 사용자의 선호를 나타내는  $\{1, 2\}$ 에 대한 확률 분포(probability distribution)이다.
- 사용자가 한 구간을 더 선호하는 경우,  $\mu$ 는 해당 구간에 모든 확률 질량(probability mass)을 할당한다.
- 두 구간이 동일하게 선호된다고 표시된 경우에는,

$$\mu = (0.5, 0.5)$$

이다.

## 4.2 Fitting reward function

추정된 보상 함수(estimated reward function)  $\hat{r}$ 는 인간의 판단(human judgments)을 설명하는 잠재 변수(latent variable), 즉 선호 함수(preference function)  $\hat{P}$ 로 볼 수 있다. 우리는 인간이 trajectory 구간  $\tau_1$ 을  $\tau_2$ 보다 선호할 확률을 다음과 같이 모델링한다:

$$\hat{P}[\tau_1 \succ \tau_2] = \frac{\exp(\sum \hat{r}(\mathbf{s}_t^1, \mathbf{a}_t^1))}{\exp(\sum \hat{r}(\mathbf{s}_t^1, \mathbf{a}_t^1)) + \exp(\sum \hat{r}(\mathbf{s}_t^2, \mathbf{a}_t^2))}, \quad (1)$$

여기서 합(summation)은 각 trajectory 구간 내부의 모든 timestep에 대해 수행된다.

$\hat{r}$ 을 학습시키기 위해, 우리는 예측된 선호(predicted preference)와 실제 인간이 라벨링한 선호 분포(human-labeled preference distribution)  $\mu$  사이의 크로스 엔트로피 손실(cross-entropy loss)을 최소화한다:

$$\mathcal{L}(\hat{r}) = - \sum_{(\tau_1, \tau_2, \mu) \in \mathcal{D}} \left[ \mu(1) \log \hat{P}[\tau_1 \succ \tau_2] + \mu(2) \log \hat{P}[\tau_2 \succ \tau_1] \right] \quad (2)$$

## 4.3 Implementation

이제 RLHF를 단계별로 구현해보자:

1. `cas4160/networks/reward_predictor.py`: 보상 모델(reward model)을 학습시키기 위해 이 파일에 있는 모든 TODO를 구현하시오.
2. `cas4160/agents/pg_agent.py`: 이 코드는 PPO를 구현하고 있지만, 위에서 구현한 보상 함수를 사용하도록 단 한 줄의 코드만 수정하면 된다.
3. `cas4160/scripts/run_hw7.py`: 사람이 주석(annotation)한 triplet을 replay buffer에 저장하고, 이를 보상 모델 학습에 사용하며, PPO 에이전트에 보상 신호(reward signal)를 제공하도록 구현해야 한다. 이 파일의 모든 TODO를 구현하시오.

# 5 Experiments

## 5.1 PointMaze with synthetic preferences

- 이 과제는 여러분의 코드를 디버깅하고 RLHF 및 피드백 메커니즘을 이해하는 데 도움을 주기 위해 설계되었다. 미로(maze) 안에는 초록색 공(green ball)과 빨간색 목표 지점(red target)이 존재한다. 목표는 초록색

공이 빨간색 목표 지점 쪽으로 이동하도록 만드는 것이다. 여러분은 초록색 공의 움직임을 바탕으로 목표 지점에 더 도달할 가능성이 높아 보이는 trajectory를 “선호(prefer)” 하는 방식으로 피드백을 제공해야 한다.

- 먼저 synthetic preference를 사용하여 보상 모델(reward model)을 학습시켜보자. 인간의 피드백을 받는 대신, 환경의 보상(environment reward)을 사용하여 synthetic preference를 생성하고, 이를 학습에 사용한다. `syn_prefs`가 **True**로 설정되었을 때 `run_hw7.py`에서 이 과정이 어떻게 동작하는지 살펴보자. 아래의 `bash` 파일을 실행하여 `PointMaze_UMazeDense-v3` 환경에서 synthetic preference를 사용해 모델을 학습하시오.

```
sh pointmaze_syn_prefs.sh
```

- 학습이 완료되면 TensorBoard를 열어 학습 결과를 확인하시오. 또한 rollout gif 역시 TensorBoard에서 다운로드할 수 있다는 점에 유의하시오.
- **질문 (Question):** 학습 결과와 evaluation rollout을 확인하시오. 학습된 에이전트는 어떤 성능을 보였는가? 보상(reward)과 미로 구조(maze structure)를 기반으로 왜 결과가 최적이지 아닌지 설명하시오. 또한 Figure 2에 rollout frame을 첨부하시오.

답변 (Answer):

설명을 작성하시오.

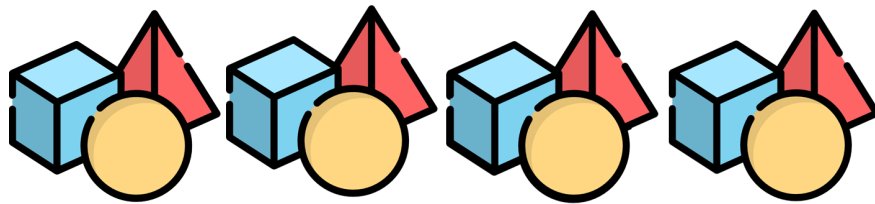


Figure 2: Synthetic preference를 사용한 point maze rollout. 이 부분을 여러분의 결과 이미지로 교체하고, 캡션을 작성하시오.

## 5.2 PointMaze with human preferences

- 이번에는 여러분의 피드백을 사용하여 미로를 더욱 효과적으로 해결해보자. 아래의 `bash` 스크립트는 trajectory annotation GUI가 포함된 코드를 실행한다.

```
sh pointmaze.sh
```

- `bash` 파일을 실행하면 터미널에서 다음과 같은 출력이 나타난다:

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://***.***.***.***:5000
Press CTRL+C to quit
Using GPU id 0

***** Iteration 0 *****

Collecting video rollouts for feedback...
```

이는 학습(training)이 시작되었음을 의미한다. 이제 웹 브라우저에서 127.0.0.1:5000 주소로 접속하여 preference feedback을 제공할 수 있다. 만약 VESSL AI를 사용 중이라면, TensorBoard 포트를 열었던 것과 동일하게 metadata panel로 이동하여 5000 포트를 클릭해야 한다. 그러면 웹 브라우저에 Figure 3와 같은 GUI 인터페이스가 표시된다. 버튼을 클릭하거나 키보드의 방향키(arrow keys)를 사용하여 피드백을 제공할 수 있다.



Figure 3: 인간 선호 피드백(human preference feedback)을 위한 웹 GUI.

- 학습이 완료되면 TensorBoard를 사용하여 결과를 시각화할 수 있다. 이 과제는 매우 단순하기 때문에 학습은 약 10분 정도 내에 완료된다. 합리적인 피드백(reasonable feedback)을 제공했다면, 초록색 공이 오류 없이 부드럽게 빨간 목표 지점(red goal)을 향해 이동하는 모습을 확인할 수 있을 것이다.

다음은 피드백을 제공할 때의 몇 가지 팁이다:

- 여러분이 보는 영상은 policy rollout 전체 trajectory가 아니다. 학습 속도를 높이기 위해 마지막 몇 초만 보여준다.
- 실험 동안 내부적으로 구체적인 기준(criteria)을 정하고, 일관된 피드백(consistent feedback)을 제공하라.
- 처음부터 완벽할 필요는 없다. 목표 지점을 향해 아주 조금이라도 움직이는 trajectory를 선택하라. 시간이 지나면서 점진적으로 성능이 향상될 것이다.
- 꼭 필요한 경우가 아니라면 'Neutral' 선택을 피하라. 아주 작은 차이도 큰 변화를 만들어낼 수 있다. 두 trajectory를 정말 구별하기 어려울 때만 'Neutral'을 선택하라.
- 학습된 모델이 의도한 목표를 따라갈 가능성이 전혀 보이지 않는다면, 학습을 다시 시작하라.
- 동일한 행동(action)을 수행했다면, 더 빠르게 수행한 trajectory가 더 우수하다.

**질문 (Question).** Figure 4에 학습된 에이전트의 성공적인 rollout frame을 첨부하시오. 또한 왜 인간 피드백(human feedback)이 이 과제에 유용한지 설명하고, 정책 학습(policy learning)에서 RLHF가 필요하거나 특히 도움이 되는 조건에 대해 논의하시오.

**답변 (Answer):**

**설명을 작성하시오.**

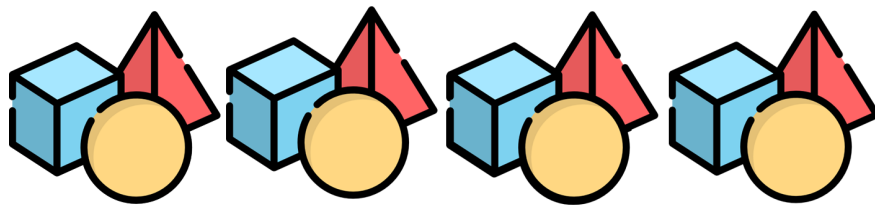


Figure 4: Human preference를 사용한 point maze rollout. 이 부분을 여러분의 결과 이미지로 교체하고, 캡션을 작성하시오.

### 5.3 Hopper Backflip

좋다. 하지만 이것은 그렇게 놀라운 결과는 아니다. 이제 조금 더 흥미로운 것을 시도해보자. 이번 섹션에서는 이전에 다양한 알고리즘을 테스트했던 Hopper 환경을 사용하여 백플립(backflip)을 시도해볼 것이다. Hopper가 백플립을 수행하도록 만드는 보상 함수(reward function)를 직접 설계한다고 상상해보라 — 꽤 어려워 보이지 않는가? 하지만 RLHF를 사용하면, 여러분의 약간의 고품질 피드백(quality feedback)과 조금 더 긴 학습 시간만으로 Hopper가 백플립을 수행하도록 만들 수 있다!

- Hopper-v5 환경에서 모델을 학습시키시오. 아래 bash 스크립트는 실행 디렉토리로 이동한 뒤 trajectory annotation GUI가 포함된 코드를 실행한다.

```
sh hopper.sh
```

- 이 실험은 5 step마다 인간 피드백(human feedback)을 요청하면서 총 100번의 optimization step을 수행한다. 전체 실험은 VESSL AI 기준으로 약 1시간 20분 정도 소요된다.
- 과제 폴더에는 Hopper가 백플립을 학습하는 예시를 보여주기 위한 *hopper\_flip.mp4* 파일이 포함되어 있다. 여러분은 해당 영상을 참고하여 피드백을 제공할 수 있다. 여러분의 prompting 능력을 보여주어라!
- Hopper가 한 번의 뒤쪽 회전(backward revolution)을 완료하면 백플립이 성공한 것으로 간주한다. 동작 도중 지면(grounding)과 접촉하는 것은 허용된다.

**질문 (Question).** 저장된 영상(saved video)에서 몇 개의 frame을 추출하여, 학습된 정책(policy)의 rollout을 Figure 5에 첨부하시오. 또한 여러분의 정책의 Eval\_AverageReturn 그래프를 Figure 6에 그리시오. 마지막으로, 마지막 성공 rollout 영상을 [Student\_ID]\_Hopper\_Backflip.gif 이름으로 제출 zip 파일 안에 포함하시오.



Figure 5: Hopper backflip rollout. 이 부분을 여러분의 결과 이미지로 교체하고, 캡션을 작성하시오.

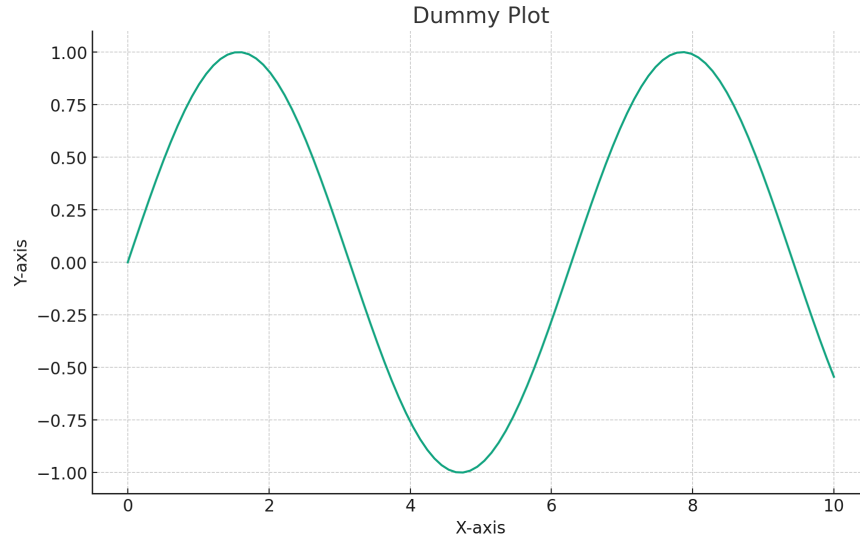


Figure 6: Hopper backflip average return. 이 부분을 여러분의 결과 그래프로 교체하고, 캡션을 작성하시오.

## 6 Submission

Please submit the code, a backflip gif, and the “**report**” in a single zip file, hw7\_[YourStudentID].zip. The zip file should be smaller than 50MB. The structure of the submission file should be:

```
hw7_[YourStudentID].zip
├── hw7_[YourStudentID].pdf
├── [YourStudentID]_Hopper_Backflip.gif
├── cas4160
│   ├── agents
│   │   └── pg_agent.py
│   ├── networks
│   │   ├── reward_predictor.py
│   │   └── ...
│   └── ...
└── ...
```

We hope you enjoy this assignment, and thank you for all your hard work throughout the semester. We also hope these homework assignments have deepened your understanding of reinforcement learning!